Cahier des charges : StudyFlow

Application web de gestion de projet pour équipes étudiantes (type mini-Trello / Notion)

Nom du projet	StudyFlow
Type	Application web full-stack
Technologie principale	Next.js (App Router) + TypeScript
Équipe	7 développeurs, 1 module chacun
Version du document	1.0

Table des matières

1. Présentation du projet
2. Périmètre
3. Utilisateurs cibles
4. Rôles et permissions
5. Choix techniques
6. Architecture et conventions
7. Modèle de données
8. Spécifications fonctionnelles par module
9. Exigences non fonctionnelles
10. Organisation de l'équipe
11. Planning prévisionnel
12. Livrables
13. Risques et parades
14. Critères de validation
15. Annexes

1. Présentation du projet

1.1 Contexte

Les étudiants travaillent de plus en plus en groupe : projets de cours, projets tutorés, associations, hackathons. Leur organisation repose souvent sur un empilement d'outils mal reliés : un groupe WhatsApp pour discuter, un Google Drive pour les fichiers, un tableur ou des notes pour les tâches, et un calendrier personnel pour les échéances. Les informations se perdent, les responsabilités sont floues, et personne n'a de vue d'ensemble sur l'avancement.

Les outils professionnels (Trello, Notion, Jira, Asana) répondent à ce besoin, mais ils sont soit trop complexes, soit payants dès qu'on dépasse les fonctions de base, soit inadaptés au contexte étudiant (notion de matière, de rendu, de soutenance, d'évaluation par un enseignant).

1.2 Problématique

> Comment permettre à une équipe d'étudiants de planifier, répartir, suivre et communiquer autour d'un projet dans un seul outil simple, gratuit et adapté à leur réalité ?

1.3 Objectifs

Objectifs principaux

- Centraliser dans une seule application la planification, les tâches, les échéances, la communication et les fichiers d'un projet.
- Donner à chaque membre une vision claire de ce qu'il doit faire et pour quand.

- Donner à l'équipe (et à un enseignant référent) une vue objective de l'avancement.

Objectifs pédagogiques (pour l'équipe de développement)

- Maîtriser Next.js (App Router, Server Components, Server Actions, Route Handlers, middleware).
- Concevoir et exploiter une base de données relationnelle avec un ORM.
- Mettre en œuvre de l'authentification, de l'autorisation, du temps réel et de l'upload de fichiers.
- Travailler en équipe de 7 : Git, revue de code, intégration continue, conventions partagées.

1.4 Indicateurs de réussite

- Une équipe de 5 étudiants peut créer un projet, inviter ses membres, créer 20 tâches et les répartir en moins de 10 minutes sans aide.
- Tous les parcours critiques sont utilisables sur ordinateur et sur mobile.
- Les 7 modules sont intégrés dans une application unique, déployée et accessible par une URL publique.

---

2. Périmètre

2.1 Inclus (version 1.0)

- Inscription, connexion, gestion du profil, création d'équipes, invitations, rôles.
- Plusieurs projets par équipe, chacun avec son tableau Kanban.
- Création, édition, assignation et suivi de tâches détaillées.
- Calendrier des échéances (vue mois / semaine / liste).
- Chat par équipe et par projet, en temps réel.
- Dépôt, organisation et consultation de fichiers.
- Statistiques d'avancement et export de rapports.
- Notifications dans l'application.

2.2 Exclu (hors périmètre de la v1.0)

- Appels audio / vidéo.
- Intégrations tierces (Google Calendar, GitHub, Slack), sauf export iCal simple.
- Facturation, abonnement, paiement.
- Édition collaborative de documents en temps réel (type Google Docs).
- Mode hors-ligne.

2.3 Évolutions possibles (bonus si le temps le permet)

- Notifications par e-mail et résumé hebdomadaire.
- Mode sombre.
- Modèles de projet (template « projet de fin d'année », « sprint de 2 semaines »).
- Diagramme de Gantt.
- Application installable (PWA).

---

3. Utilisateurs cibles

3.1 Personas

Persona	Profil	Besoin principal
Léa, cheffe de projet	Étudiante de L3, organise le projet de groupe	Répartir le travail, relancer, avoir une vue d'ensemble
Karim, contributeur	Étudiant dans l'équipe, travaille aussi à côté	Savoir clairement ce qu'il doit faire et pour quand, sans noyade d'informations
Mme Dupont, enseignante	Encadre plusieurs groupes	Suivre l'avancement de chaque groupe sans micro-gérer

référente		
Sami, étudiant multi-projets	Membre de 3 équipes différentes (cours, asso, stage)	Une vue unifiée de toutes ses tâches et échéances

3.2 Parcours type

1. Léa crée un compte, puis crée l'équipe « Projet Web S5 » et invite 4 camarades par lien d'invitation.
2. Elle crée un projet « Application de réservation » avec un tableau Kanban (À faire, En cours, En relecture, Terminé).
3. Elle crée les tâches, les assigne, fixe des échéances.
4. Karim se connecte, voit ses tâches, déplace une carte vers « En cours », joint un fichier, commente.
5. L'équipe discute dans le chat du projet.
6. La veille du rendu, Léa consulte les statistiques et exporte un rapport d'avancement.

---

4. Rôles et permissions

4.1 Rôles au niveau d'une équipe

Rôle	Description
Propriétaire	Créateur de l'équipe. Tous les droits, dont la suppression de l'équipe et le transfert de propriété.
Administrateur	Gère les membres, les projets et les paramètres.
Membre	Crée et modifie des tâches, participe au chat, dépose des fichiers.
Observateur	Lecture seule (ex. enseignant référent). Peut commenter si autorisé.

4.2 Matrice des permissions

Action	Propriétaire	Admin	Membre	Observateur
Supprimer l'équipe	✔	✘	✘	✘
Inviter / retirer un membre	✔	✔	✘	✘
Changer le rôle d'un membre	✔	✔ (sauf propriétaire)	✘	✘
Créer / archiver un projet	✔	✔	✘	✘
Créer / modifier des colonnes	✔	✔	✔	✘
Créer / modifier / déplacer des tâches	✔	✔	✔	✘
Supprimer une tâche	✔	✔	Seulement ses tâches	✘
Écrire dans le chat	✔	✔	✔	✔ (lecture seule par défaut, option)
Déposer un fichier	✔	✔	✔	✘
Supprimer un fichier	✔	✔	Seulement les siens	✘
Voir les statistiques	✔	✔	✔	✔
Exporter un rapport	✔	✔	✔	✔

> Règle d'or : **toute vérification de permission est faite côté serveur**. L'interface masque les boutons interdits par confort, mais ne constitue jamais une protection.

---

5. Choix techniques

Domaine	Choix recommandé	Alternative
Framework	**Next.js 15+ (App Router)**	n/a (imposé)
Langage	**TypeScript** (mode strict)	n/a
Base de données	**PostgreSQL** (Neon ou Supabase en hébergement gratuit)	SQLite en local pour le développement
ORM	**Prisma**	Drizzle ORM
Authentification	**Auth.js (NextAuth v5)** : e-mail/mot de passe + Google/GitHub	Better Auth, Clerk

Validation	**Zod** (schémas partagés client/serveur)	Valibot
UI	**Tailwind CSS + shadcn/ui**	Mantine, Chakra
Drag & drop	**dnd-kit**	@hello-pangea/dnd
Calendrier	**FullCalendar** ou **react-big-calendar**	Calendrier maison avec date-fns
Temps réel	**Pusher / Ably / Supabase Realtime**	Serveur Socket.IO séparé (Railway, Render)
Stockage de fichiers	**UploadThing**, **Supabase Storage** ou **AWS S3 / Cloudflare R2**	Stockage disque local (dev uniquement)
Graphiques	**Recharts**	Chart.js, Nivo
Export PDF	**@react-pdf/renderer** ou génération par impression navigateur	jsPDF
Données côté client	**TanStack Query** + Server Actions	SWR
Formulaires	**React Hook Form** + Zod	n/a
Dates	**date-fns**	Day.js
Tests	**Vitest** (unitaires), **Playwright** (end-to-end)	Jest, Cypress
Qualité	**ESLint, Prettier, Husky + lint-staged**	Biome
CI/CD	**GitHub Actions**	GitLab CI
Déploiement	**Vercel** (app) + service de temps réel externe	Railway, Render

> **Point d'attention : le temps réel.** Vercel étant « serverless », il ne conserve pas de connexions WebSocket persistantes. Deux solutions : (1) utiliser un service tiers (Pusher, Ably, Supabase Realtime), simple et gratuit à petite échelle ; (2) héberger un petit serveur Socket.IO séparé. La décision doit être prise lors de la semaine 1 par le responsable du Module 5.

---

6. Architecture et conventions

6.1 Arborescence cible

```
studyflow/
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
├── src/
│   ├── app/
│   │   ├── (auth)/                # Module 1 : login, register, reset
│   │   ├── (app)/
│   │   │   ├── layout.tsx         # Layout connecté (sidebar, header)
│   │   │   ├── dashboard/         # Vue d'accueil (agrège les modules)
│   │   │   ├── teams/             # Module 1
│   │   │   │   └── [teamId]/
│   │   │   │       ├── settings/
│   │   │   │       ├── chat/      # Module 5
│   │   │   │       └── projects/
│   │   │   │           └── [projectId]/
│   │   │   │               ├── board/      # Module 2
│   │   │   │               ├── tasks/      # Module 3
│   │   │   │               ├── calendar/   # Module 4
│   │   │   │               ├── chat/       # Module 5
│   │   │   │               ├── files/      # Module 6
│   │   │   │               └── stats/      # Module 7
│   │   │   └── me/                # Mes tâches, mon calendrier, mon profil
│   │   └── api/                   # Route Handlers (webhooks, upload, realtime auth)
│   ├── modules/
│   │   ├── auth-teams/            # Module 1
│   │   ├── kanban/                # Module 2
│   │   ├── tasks/                 # Module 3
│   │   ├── calendar/              # Module 4
│   │   ├── chat/                  # Module 5
│   │   ├── files/                 # Module 6
│   │   └── stats/                 # Module 7
│   │       ├── components/
│   │       ├── actions.ts         # Server Actions
│   │       ├── queries.ts         # Lectures en base
│   │       ├── schemas.ts         # Schémas Zod
│   │       └── types.ts
│   ├── components/ui/             # Design system partagé (shadcn)
│   ├── lib/                       # prisma.ts, auth.ts, permissions.ts, utils
│   └── middleware.ts
├── tests/
├── .github/workflows/ci.yml
└── README.md
```

Principe : chaque module vit dans `src/modules/<nom>` (logique, composants, requêtes) et expose ses pages dans `src/app/...`. Cela limite les conflits Git, car chaque développeur travaille presque uniquement dans son dossier.

6.2 Conventions de code

- **Nommage** : fichiers en `kebab-case`, composants en `PascalCase`, fonctions en `camelCase`, tables Prisma en `PascalCase` singulier.
- **Langue** : code, commits et branches en anglais ; interface utilisateur en français (prévoir l'internationalisation si possible).
- **Données** : toute entrée utilisateur est validée par Zod côté serveur.
- **Mutations** : via Server Actions, qui appellent `assertPermission()` (voir `lib/permissions.ts`) avant toute écriture.
- **Erreurs** : format de retour standard `{ ok: true, data } | { ok: false, error: { code, message } }`.
- **Composants serveur par défaut** ; `"use client"` uniquement quand l'interactivité l'exige.
- **Accessibilité** : composants navigables au clavier, labels, contrastes AA.

6.3 Contrat entre modules

Pour que les 7 modules s'assemblent sans douleur, ces points sont **figés dès la semaine 1** :

1. **Schéma Prisma** : propriété d'un module précis (voir §7), modifications par pull request validée par le propriétaire.
2. **`lib/permissions.ts`** (Module 1) : `getCurrentUser()`, `requireTeamRole(teamId, minRole)`, `requireProjectAccess(projectId)`. Tous les modules l'utilisent.
3. **Positionnement des cartes et colonnes** : champ `position` de type `String` avec *fractional indexing* (bibliothèque `fractional-indexing`), ce qui évite de renuméroter toute la colonne à chaque déplacement.
4. **Événements temps réel** : convention de nommage `<ressource>.<action>`, par exemple `task.moved`, `task.updated`, `message.created`, `file.uploaded`. Canaux : `team-{id}` et `project-{id}`.
5. **Journal d'activité** : tout module appelle `logActivity({ projectId, userId, type, payload })` après une mutation (alimente le Module 7 et les notifications).
6. **Design system** : composants dans `components/ui`, tokens (couleurs, espacements) définis dans Tailwind. Aucune couleur écrite en dur.

---

7. Modèle de données

7.1 Entités principales

Entité	Champs principaux	Propriétaire
User	id, name, email, passwordHash, image, createdAt	M1
Account / Session	tables Auth.js (OAuth, sessions)	M1
Team	id, name, description, slug, ownerId, createdAt	M1
TeamMember	id, teamId, userId, role (`OWNER` , `ADMIN` , `MEMBER` , `VIEWER`), joinedAt	M1
Invitation	id, teamId, email?, token, role, expiresAt, usedAt	M1
Project	id, teamId, name, description, color, startDate, endDate, archivedAt	M2
Column	id, projectId, name, position, wipLimit?, isDone (colonne « terminé »)	M2
Task	id, projectId, columnId, title, description, position, priority, status, dueDate, startDate, estimate, createdById, createdAt, completedAt	M3
TaskAssignee	taskId, userId	M3
Label	id, projectId, name, color	M3
TaskLabel	taskId, labelId	M3
Subtask	id, taskId, title, done, position	M3
Comment	id, taskId, authorId, content, createdAt, editedAt	M3
Milestone	id, projectId, title, date, description	M4
Channel	id, teamId, projectId?, name, type (`TEAM` , `PROJECT`)	M5
Message	id, channelId, authorId, content, replyToId?, createdAt, editedAt, deletedAt	M5

MessageRead	channelId, userId, lastReadAt	M5
File	id, projectId, taskId?, folderId?, uploaderId, name, mimeType, size, storageKey, version, createdAt	M6
Folder	id, projectId, parentId?, name	M6
Activity	id, projectId, userId, type, payload (JSON), createdAt	Transverse (M7 lit)
Notification	id, userId, type, payload (JSON), readAt, createdAt	Transverse (M1 héberge)

7.2 Relations clés

```
User ──< TeamMember >── Team ──< Project ──< Column ──< Task
                                  │                    ├──< TaskAssignee >── User
                                  │                    ├──< Subtask
                                  │                    ├──< Comment
                                  │                    ├──< TaskLabel >── Label
                                  │                    └──< File
                                  ├──< Milestone
                                  ├──< Folder ──< File
                                  ├──< Channel ──< Message
                                  └──< Activity
```

7.3 Règles d'intégrité

- Supprimer une équipe supprime en cascade ses projets, tâches, fichiers (y compris dans le stockage) et messages.
- Un utilisateur ne peut être qu'une seule fois membre d'une même équipe (contrainte unique `teamId + userId`).
- Une équipe a toujours au moins un propriétaire.
- Une tâche appartient à exactement une colonne ; les colonnes d'un projet sont ordonnées par `position`.
- Les suppressions de messages et de tâches peuvent être « douces » (`deletedAt`) pour conserver l'historique.
- Index recommandés : `Task(projectId, columnId, position)`, `Task(dueDate)`, `Message(channelId, createdAt)`, `Activity(projectId, createdAt)`.

---

8. Spécifications fonctionnelles par module

Légende de priorité : **P1** = indispensable (MVP), **P2** = important, **P3** = bonus.

Module 1 : Authentification et gestion des équipes

Objectif. Permettre à chacun de se connecter de façon sécurisée, de créer ou rejoindre des équipes, et fournir à toute l'application le socle d'identité et de permissions.

Responsabilités transverses (socle commun). Ce module livre dès la semaine 1 le layout connecté, le middleware de protection des routes, et `lib/permissions.ts`.

Fonctionnalités

Réf.	Fonctionnalité	Priorité
1.1	Inscription par e-mail / mot de passe (validation, mot de passe haché avec bcrypt ou argon2)	P1
1.2	Connexion / déconnexion, session persistante	P1
1.3	Connexion via Google et/ou GitHub (OAuth)	P2
1.4	Réinitialisation du mot de passe par e-mail	P2
1.5	Profil utilisateur : nom, photo, mot de passe, suppression du compte	P1
1.6	Création d'une équipe (nom, description)	P1
1.7	Invitation de membres par lien (jeton à durée limitée) et/ou par e-mail	P1
1.8	Gestion des membres : liste, changement de rôle, retrait, départ volontaire	P1
1.9	Paramètres d'équipe : renommer, transférer la propriété, supprimer	P2
1.10	Sélecteur d'équipe dans la barre latérale (multi-équipes)	P1
1.11	Centre de notifications (cloche, liste, marquage comme lu)	P2
1.12	Protection des routes par middleware, redirection vers la connexion	P1

Règles de gestion

- Mot de passe : 8 caractères minimum, avec vérification de force.
- Un lien d'invitation expire au bout de 7 jours et peut être révoqué.
- Un propriétaire ne peut pas quitter l'équipe sans transférer la propriété.
- Un e-mail ne peut être associé qu'à un seul compte.
- Limitation du nombre de tentatives de connexion (anti brute-force).

Écrans

- `/login`, `/register`, `/forgot-password`, `/reset-password`
- `/teams` (liste de mes équipes), `/teams/new`
- `/teams/[teamId]/settings` (membres, invitations, paramètres)
- `/invite/[token]` (acceptation d'invitation)
- `/me` (profil)

Critères d'acceptation

- Un utilisateur non connecté qui accède à une page protégée est redirigé vers `/login`, puis ramené sur sa page après connexion.
- Un membre simple ne peut ni inviter ni retirer quelqu'un, y compris en appelant directement l'action côté serveur.
- Un lien d'invitation expiré affiche un message clair.

Dépendances

Aucune en entrée. Tous les autres modules en dépendent.

Module 2 : Tableaux Kanban (drag & drop)

Objectif. Offrir une vue visuelle et interactive du travail de l'équipe, organisée en colonnes représentant les étapes d'avancement.

Fonctionnalités

Réf.	Fonctionnalité	Priorité
2.1	Création d'un projet dans une équipe (nom, description, couleur, dates)	P1
2.2	Liste des projets de l'équipe, archivage / restauration	P1
2.3	Création automatique de 4 colonnes par défaut (À faire, En cours, En relecture, Terminé)	P1
2.4	Ajout, renommage, suppression, réordonnancement des colonnes (drag & drop)	P1
2.5	Déplacement des cartes par drag & drop, entre colonnes et dans une colonne	P1
2.6	Positionnement persistant (rechargement = même ordre) via `position`	P1
2.7	Mises à jour optimistes (la carte bouge immédiatement, rollback en cas d'erreur)	P1
2.8	Création rapide d'une carte depuis le bas d'une colonne	P1
2.9	Carte résumée : titre, étiquettes, assignés (avatars), échéance, compteur de sous-tâches, de commentaires et de pièces jointes	P1
2.10	Filtres : par assigné, étiquette, priorité, échéance, recherche texte	P2
2.11	Limite de cartes par colonne (WIP limit) avec avertissement visuel	P3
2.12	Synchronisation temps réel : le déplacement d'un collègue apparaît à l'écran	P2
2.13	Support tactile (glisser au doigt sur mobile) et clavier (accessibilité)	P2
2.14	Choix de la couleur de colonne / de la couverture de carte	P3

Règles de gestion

- Déplacer une carte dans la colonne marquée `isDone` renseigne `completedAt` ; la sortir de cette colonne le remet à `null`.
- Supprimer une colonne non vide demande de choisir où déplacer ses cartes (ou de les supprimer explicitement).
- Seuls les rôles Membre et supérieurs peuvent déplacer des cartes.
- Chaque déplacement alimente le journal d'activité (`task.moved`, avec colonne d'origine et de destination).

Écrans

- `/teams/[teamId]/projects` (liste/grille des projets)
- `/teams/[teamId]/projects/[projectId]/board` (tableau Kanban)

Critères d'acceptation

- Déplacer une carte entre deux colonnes puis recharger la page conserve la position exacte.
- Deux utilisateurs déplaçant des cartes différentes en même temps ne se gênent pas ; en cas de conflit sur la même carte, la dernière écriture gagne et l'interface se resynchronise.
- Le tableau reste fluide avec 100 cartes réparties sur 6 colonnes.

Dépendances

M1 (permissions), M3 (modèle `Task` et fenêtre de détail de tâche), M5/temps réel (infrastructure d'événements).

Module 3 : Gestion des tâches et assignations

Objectif. Fournir le cœur « métier » de l'application : une tâche riche, assignable, commentable, avec sa propre vue détaillée, et une vue personnelle « Mes tâches ».

Responsabilité d'interface. Ce module est propriétaire du modèle `Task` et expose les fonctions de lecture/écriture que M2, M4 et M7 consomment (`createTask` , `updateTask` , `moveTask` , `listTasks(filters)`).

Fonctionnalités

Réf.	Fonctionnalité	Priorité
3.1	Création d'une tâche : titre, description (Markdown), priorité (basse, normale, haute, urgente), échéance	P1
3.2	Fenêtre de détail de tâche (modale ou panneau latéral, avec URL partageable)	P1
3.3	Assignation à un ou plusieurs membres de l'équipe	P1
3.4	Étiquettes personnalisables par projet (nom + couleur)	P1
3.5	Sous-tâches (checklist) avec barre de progression	P1
3.6	Commentaires sur une tâche, avec mentions `@nom`	P2
3.7	Historique des modifications d'une tâche (qui a changé quoi, quand)	P2
3.8	Vue liste/tableau de toutes les tâches du projet, avec tri, filtres et recherche	P1
3.9	Page « Mes tâches » : toutes mes tâches, tous projets confondus, groupées par échéance	P1
3.10	Notification à l'assigné lors d'une assignation, d'une mention ou d'un commentaire	P2
3.11	Estimation de charge (heures ou points) et temps passé	P3
3.12	Dépendances entre tâches (« bloquée par »)	P3
3.13	Duplication, archivage et suppression d'une tâche	P2
3.14	Actions groupées (assigner, changer la priorité, déplacer plusieurs tâches)	P3

Règles de gestion

- Seuls les membres de l'équipe peuvent être assignés à une tâche du projet.
- Une tâche en retard (échéance dépassée et non terminée) est signalée visuellement.
- Le titre est obligatoire (1 à 120 caractères) ; la description est limitée à 10 000 caractères.
- La suppression d'une tâche supprime ses sous-tâches, commentaires et le lien vers ses fichiers (les fichiers eux-mêmes sont conservés dans le Module 6 avec `taskId = null` , ou supprimés selon choix de l'utilisateur).

Écrans

- `/teams/[teamId]/projects/[projectId]/tasks` (liste/tableau)
- `/teams/[teamId]/projects/[projectId]/tasks/[taskId]` (détail, aussi affiché en modale depuis le Kanban)
- `/me/tasks` (mes tâches)

Critères d'acceptation

- Assigner une tâche à un utilisateur la fait apparaître dans sa page « Mes tâches » et déclenche une notification.
- Cocher toutes les sous-tâches ne termine pas automatiquement la tâche, mais le propose.
- Les filtres de la vue liste sont conservés dans l'URL (partageables).

Dépendances

M1 (membres, permissions), M2 (colonnes), M6 (pièces jointes), M5 (notifications temps réel, optionnel).

Module 4 : Calendrier et échéances

Objectif. Offrir une vision temporelle du projet pour anticiper les deadlines et équilibrer la charge de travail.

Fonctionnalités

Réf.	Fonctionnalité	Priorité
4.1	Vue calendrier mensuelle affichant les échéances des tâches	P1
4.2	Vues semaine et liste (agenda)	P2
4.3	Calendrier par projet et calendrier personnel (tous mes projets)	P1
4.4	Clic sur un événement : ouverture du détail de la tâche (M3)	P1
4.5	Modification de l'échéance par glisser-déposer d'une tâche vers un autre jour	P2
4.6	Création d'une tâche en cliquant sur un jour	P2
4.7	Jalons (milestones) : dates clés du projet (rendu, soutenance, livraison)	P1
4.8	Code couleur par priorité, par projet ou par assigné	P2
4.9	Mise en évidence des tâches en retard et des échéances du jour	P1
4.10	Filtres : par projet, assigné, étiquette	P2
4.11	Rappels : notification 24 h et 1 h avant l'échéance (tâche planifiée / cron)	P2
4.12	Export iCal (`.ics`) et lien d'abonnement pour Google Calendar / Apple Calendar	P3
4.13	Vue de charge : nombre de tâches par personne et par semaine (détection de surcharge)	P3
4.14	Plage de dates (début + fin) pour une tâche, affichée en barre sur plusieurs jours	P3

Règles de gestion

- Les tâches sans échéance n'apparaissent pas dans le calendrier (elles sont listées dans un panneau « Non planifiées »).
- Les jalons sont créés par les administrateurs et propriétaires uniquement.
- Les dates sont stockées en UTC et affichées dans le fuseau horaire de l'utilisateur.
- Le premier jour de la semaine est le lundi.

Écrans

- `/teams/[teamId]/projects/[projectId]/calendar`
- `/me/calendar` (calendrier personnel)

Critères d'acceptation

- Modifier l'échéance d'une tâche dans le détail met à jour le calendrier sans rechargement manuel.
- Un jalon créé est visible par tous les membres du projet.
- Un événement du calendrier est lisible et utilisable sur un écran de 375 px de large.

Dépendances

M1 (identité, fuseau), M3 (tâches et échéances), M7 (indicateurs de retard, optionnel).

Module 5 : Chat d'équipe en temps réel

Objectif. Faciliter la communication de l'équipe au sein même de l'outil, et fournir au reste de l'application l'infrastructure de temps réel.

Responsabilité d'infrastructure. Ce module met en place (semaine 1-2) le service d'événements temps réel et en expose une API simple `publish(channel, event, payload)` / `useChannel(channel, handlers)` que M2, M3 et M6 peuvent utiliser.

Fonctionnalités

Réf.	Fonctionnalité	Priorité
5.1	Salon général par équipe et salon par projet (créés automatiquement)	P1
5.2	Envoi et réception de messages en temps réel	P1
5.3	Historique persistant avec chargement progressif (pagination infinie vers le haut)	P1
5.4	Indicateur « en train d'écrire… »	P2
5.5	Présence : qui est en ligne	P2
5.6	Messages non lus : badge par salon et marqueur « nouveaux messages »	P2
5.7	Modification et suppression de ses propres messages	P2
5.8	Réponse à un message (citation)	P3
5.9	Mentions `@membre` (notification) et référence de tâche `#42` (lien vers la tâche)	P2
5.10	Mise en forme Markdown simple (gras, italique, code, liens)	P2
5.11	Réactions par émojis	P3
5.12	Messages système automatiques (ex. « Karim a terminé la tâche X ») issus du journal d'activité	P3
5.13	Envoi d'un fichier ou d'une image dans le chat (via M6)	P3
5.14	Recherche dans les messages	P3
5.15	Messages privés entre deux membres	P3

Règles de gestion

- Seuls les membres d'une équipe peuvent lire et écrire dans ses salons.
- Un message est limité à 4 000 caractères ; envoi vide interdit.
- Un message supprimé est remplacé par « Message supprimé » (suppression douce).
- Limitation de débit : 20 messages par minute et par utilisateur (anti-spam).
- L'authentification du canal temps réel est vérifiée côté serveur (un utilisateur ne peut pas s'abonner au canal d'une équipe dont il n'est pas membre).

Écrans

- `/teams/[teamId]/chat` (salon d'équipe)
- `/teams/[teamId]/projects/[projectId]/chat` (salon du projet)
- Panneau de chat rétractable, accessible depuis toutes les pages du projet

Critères d'acceptation

- Un message envoyé par un utilisateur apparaît chez les autres en moins de 1 seconde, sans rechargement.
- Après une coupure de connexion, la reconnexion récupère les messages manqués.
- Un utilisateur non membre qui tente de lire un salon reçoit une erreur 403.

Dépendances

M1 (identité, appartenance), M3 (références de tâches), M6 (fichiers dans le chat, optionnel).

Module 6 : Partage de fichiers

Objectif. Centraliser les documents du projet (cahiers des charges, maquettes, comptes rendus, rendus finaux) et les relier aux tâches.

Fonctionnalités

Réf.	Fonctionnalité	Priorité
6.1	Téléversement de fichiers par bouton ou glisser-déposer, avec barre de progression	P1
6.2	Liste des fichiers du projet (nom, type, taille, auteur, date) avec tri et recherche	P1
6.3	Téléchargement d'un fichier (lien signé à durée limitée)	P1
6.4	Aperçu intégré : images, PDF, texte / Markdown, vidéo	P2
6.5	Pièces jointes liées à une tâche (affichées dans le détail de la tâche et sur la carte)	P1
6.6	Dossiers et sous-dossiers, déplacement de fichiers	P2
6.7	Renommage et suppression (selon permissions)	P1
6.8	Versionnage : remplacer un fichier en gardant l'historique des versions	P3
6.9	Lien de partage public temporaire (optionnel, désactivable)	P3
6.10	Quota de stockage par équipe avec jauge d'utilisation	P2
6.11	Vignettes pour les images	P3
6.12	Téléversement multiple et archive ZIP du dossier à télécharger	P3

Règles de gestion

- Taille maximale par fichier : 20 Mo (configurable). Quota par équipe : 500 Mo (configurable).
- Types autorisés : documents (pdf, docx, xlsx, pptx, txt, md), images (png, jpg, gif, webp, svg), archives (zip), code source courant. Les exécutables (`.exe` , `.sh` , `.bat` , etc.) sont refusés.
- Le type MIME est vérifié côté serveur, pas seulement l'extension.
- Les fichiers ne sont **jamais** servis par une URL publique permanente : accès contrôlé par la session ou lien signé.
- Un fichier supprimé est effacé du stockage, pas seulement de la base.
- Les noms de fichiers sont nettoyés (caractères dangereux, chemins relatifs).

Écrans

- `/teams/[teamId]/projects/[projectId]/files` (explorateur de fichiers)
- Zone de pièces jointes dans le détail d'une tâche (composant exposé à M3)

Critères d'acceptation

- Un utilisateur non membre de l'équipe ne peut pas télécharger un fichier, même en connaissant son identifiant.
- Un fichier de 15 Mo est téléversé avec progression et apparaît dans la liste sans rechargement.
- Supprimer un fichier retire la pièce jointe de la tâche associée.

Dépendances

M1 (permissions), M3 (pièces jointes), M5 (partage dans le chat, optionnel).

Module 7 : Statistiques et rapports d'avancement

Objectif. Transformer les données du projet en indicateurs lisibles pour piloter le travail, repérer les blocages et préparer les points d'étape ou la soutenance.

Fonctionnalités

Réf.	Fonctionnalité	Priorité
7.1	Tableau de bord du projet : % d'avancement global, tâches totales / terminées / en retard	P1
7.2	Répartition des tâches par colonne / statut (graphique en barres ou anneau)	P1
7.3	Répartition par priorité et par étiquette	P2
7.4	Charge par membre : tâches assignées, terminées, en retard	P1
7.5	Burndown chart : travail restant en fonction du temps vs trajectoire idéale	P2
7.6	Vélocité : tâches terminées par semaine	P2
7.7	Flux cumulé (cumulative flow) ou temps moyen de traitement (lead time / cycle time)	P3
7.8	Sélecteur de période (7 jours, 30 jours, depuis le début, personnalisé)	P2
7.9	Flux d'activité récent (qui a fait quoi)	P1
7.10	Alertes : tâches bloquées depuis plus de X jours, projet en retard sur le planning, membre inactif	P3
7.11	Vue enseignant / synthèse multi-projets d'une équipe	P3
7.12	Génération d'un rapport d'avancement (résumé, graphiques, liste des tâches terminées et restantes)	P1
7.13	Export du rapport en PDF et des données en CSV	P2
7.14	Tableau de bord global de l'utilisateur (tous projets) sur la page d'accueil	P2

Règles de gestion

- L'avancement global est calculé à partir de la proportion de tâches dans la colonne « terminé » (option : pondération par estimation).
- Une tâche est « en retard » si `dueDate < aujourd'hui` et `completedAt` est vide.
- Les statistiques de charge par membre sont visibles de tous les membres (transparence), mais ne servent pas à classer ou sanctionner. Prévoir une option pour les masquer si l'équipe le souhaite.
- Les calculs lourds sont faits côté serveur (requêtes SQL agrégées) et mis en cache (`revalidate`).
- Le rapport est daté et reproductible : il porte la date et la période de génération.

Écrans

- `/teams/[teamId]/projects/[projectId]/stats` (tableau de bord)
- `/teams/[teamId]/projects/[projectId]/stats/report` (aperçu et export du rapport)
- Widget de synthèse sur `/dashboard`

Critères d'acceptation

- Déplacer une tâche en « Terminé » met à jour le pourcentage d'avancement à la prochaine consultation (ou en direct si le temps réel est branché).
- Le rapport PDF généré contient au minimum : titre du projet, période, avancement, graphique de répartition, charge par membre, tâches en retard.
- Les graphiques sont lisibles (légendes, couleurs accessibles) et s'adaptent au mobile.

Dépendances

M2/M3 (tâches, colonnes), M4 (dates et jalons), M1 (membres), journal d'activité.

---

9. Exigences non fonctionnelles

9.1 Performance

- Premier affichage d'une page principale : moins de 2,5 secondes (LCP) sur connexion standard.

- Interactions (déplacement de carte, envoi de message) : retour visuel immédiat (moins de 100 ms perçu grâce aux mises à jour optimistes).
- Requêtes paginées pour toute liste susceptible de dépasser 50 éléments.
- Pas de requêtes N+1 (utiliser `include` / `select` Prisma à bon escient).

9.2 Sécurité

- Mots de passe hachés (bcrypt/argon2), jamais journalisés.
- Cookies de session `HttpOnly`, `Secure`, `SameSite`.
- Protection CSRF (native pour les Server Actions), échappement systématique du contenu utilisateur (protection XSS) ; le Markdown est assaini avant affichage.
- Contrôle d'accès vérifié côté serveur pour **chaque** lecture et écriture (équipe, projet, ressource).
- Secrets uniquement dans les variables d'environnement, jamais dans Git. Fournir un fichier `.env.example`.
- Limitation de débit sur les routes sensibles (connexion, inscription, envoi de messages, upload).
- Dépendances surveillées (Dependabot).

9.3 Respect des données personnelles (RGPD)

- Collecte minimale (nom, e-mail, photo facultative).
- Possibilité de supprimer son compte et d'exporter ses données.
- Page de politique de confidentialité et mentions légales.
- Pas de traceur tiers sans consentement.

9.4 Accessibilité et ergonomie

- Conformité visée : WCAG 2.1 niveau AA sur les parcours principaux.
- Navigation complète au clavier (y compris le déplacement de cartes), focus visible, attributs ARIA sur les composants interactifs.
- Contraste suffisant, ne pas transmettre une information par la seule couleur.
- Responsive : ordinateur (≥ 1024 px), tablette, mobile (≥ 375 px).
- États de chargement (squelettes), états vides pédagogiques et messages d'erreur compréhensibles.

9.5 Qualité et maintenabilité

- TypeScript strict, zéro erreur ESLint, formatage Prettier automatique.
- Couverture de tests : au minimum, tests unitaires sur la logique métier critique (permissions, calcul de statistiques, positionnement) et 3 à 5 scénarios end-to-end (inscription, création d'équipe, création et déplacement d'une tâche, envoi d'un message, upload d'un fichier).
- README à jour : installation en moins de 10 minutes, variables d'environnement, scripts, architecture.
- Documentation de chaque module (une page `docs/module-X.md`).

9.6 Compatibilité

- Navigateurs : dernières versions de Chrome, Firefox, Safari et Edge.
- Node.js LTS ; gestionnaire de paquets unique pour toute l'équipe (pnpm recommandé).

9.7 Disponibilité

- Application accessible localement lors de l'évaluation.
- Jeu de données de démonstration (`prisma/seed.ts`) : 1 équipe, 5 membres, 2 projets, ~40 tâches, messages et fichiers d'exemple.

---

10. Organisation de l'équipe

10.1 Répartition des modules

Développeur	Module	Rôle complémentaire proposé
Dev 1	M1 : Auth et équipes	Responsable du socle technique (layout, permissions, middleware)

Dev 1	M1 : Auth et équipes	Responsable du socle technique (layout, permissions, middleware)
Dev 2	M2 : Kanban	Responsable du design system et de l'ergonomie
Dev 3	M3 : Tâches et assignations	Responsable du schéma de base de données et des migrations
Dev 4	M4 : Calendrier	Responsable des tests end-to-end
Dev 5	M5 : Chat temps réel	Responsable de l'infrastructure temps réel et du déploiement
Dev 6	M6 : Fichiers	Responsable de la sécurité et de l'intégration continue
Dev 7	M7 : Statistiques	Responsable de la documentation, du jeu de données de démo et du suivi du planning

10.2 Workflow Git

- Dépôt unique GitHub, branche `main` protégée (merge uniquement via pull request).
- Branche d'intégration `develop` ; branches de fonctionnalité `feat/m3-subtasks`, `fix/m2-drag-bug`.
- **Pull request obligatoire** avec au moins 1 relecture (idéalement croisée avec le module voisin), CI au vert, description claire.
- Commits suivant *Conventional Commits* (`feat:` , `fix:` , `docs:` , `refactor:` , `test:`).
- Les modifications de `schema.prisma` sont annoncées dans le canal de l'équipe avant fusion, et les migrations fusionnées rapidement pour éviter les conflits.

10.3 Rituels

- **Réunion de lancement** (semaine 0) : validation du cahier des charges, choix du nom, du design.
- **Point court 2 fois par semaine** (15 min) : où j'en suis, ce qui me bloque.
- **Revue de fin de sprint** (toutes les 2 semaines) : démonstration de ce qui fonctionne.
- **Tableau de suivi** : utiliser... StudyFlow lui-même dès qu'il est utilisable (dogfooding).

10.4 Définition de « terminé » (Definition of Done)

Une fonctionnalité est terminée lorsque :

1. Elle respecte les critères d'acceptation du module.
2. Le code est relu et fusionné via pull request.
3. Les permissions sont vérifiées côté serveur.
4. Elle est utilisable sur mobile et au clavier.
5. Les cas d'erreur et les états vides sont gérés.
6. Un test existe pour la logique critique.
7. La documentation du module est à jour.

---

11. Planning prévisionnel

Planning indicatif sur **8 semaines** (à adapter à votre calendrier réel).

Semaine	Phase	Contenu
S0	Cadrage	Validation du cahier des charges, maquettes (Figma), choix du nom, création du dépôt, CI, conventions
S1	Socle	Initialisation Next.js, Prisma, schéma initial complet, Auth.js, layout connecté, design system minimal. Choix de l'infrastructure temps réel et du stockage. Tous les modules démarrent avec des données fictives (*mocks*)
S2	Sprint 1	**MVP des modules P1** : M1 (équipes, invitations), M2 (projets, colonnes), M3 (CRUD tâches), M5 (envoi/réception de messages) en place ; M4 (calendrier mensuel), M6 (upload basique), M7 (indicateurs de base)
S3	Sprint 1 (fin)	Fin des fonctionnalités P1. **Première intégration** : bout en bout « créer équipe → projet → tâche → déplacer ». Démo interne
S4	Sprint 2	Fonctionnalités P2 : drag & drop temps réel, commentaires, mentions, aperçu de fichiers, burndown
S5	Sprint 2 (fin)	Fin des P2, notifications, filtres. Deuxième intégration. Revue de code croisée
S6	Consolidation	Bugs, performance, accessibilité, responsive, sécurité (audit des permissions), tests end-to-end
S7	Finalisation	Fonctionnalités P3 choisies, jeu de données de démo, documentation, déploiement de production
S8	Livraison	Gel du code (*code freeze*), répétition de la soutenance, rapport final, démo

Jalons clés

Jalon	Échéance	Critère
J1	Fin S1	Socle déployé, tous les modules branchés sur la base
J2	Fin S3	Parcours principal fonctionnel de bout en bout
J3	Fin S5	Toutes les fonctionnalités P1 et P2 intégrées
J4	Fin S7	Application stable, déployée, documentée
J5	S8	Soutenance

> **Conseil.** Ne laissez pas l'intégration à la fin. Chaque sprint doit se terminer par une application qui tourne avec **tous** les modules ensemble, même incomplets.

12. Livrables

1. **Code source** : dépôt GitHub propre, historique lisible, README complet.
2. **Application déployée** : URL publique + identifiants de démonstration.
3. **Base de données** : schéma Prisma, migrations, script de seed.
4. **Documentation technique** : architecture, modèle de données (diagramme), guide d'installation, une page par module.
5. **Documentation utilisateur** : guide de prise en main (captures d'écran ou courte vidéo).
6. **Maquettes** : fichier Figma ou équivalent.
7. **Rapport de projet** : organisation, difficultés rencontrées, choix techniques, bilan individuel par module.
8. **Présentation de soutenance** : support et démonstration en direct (prévoir une vidéo de secours).
9. **Rapport de tests** : tests automatisés et résultats.

13. Risques et parades

Risque	Probabilité	Impact	Parade
Conflits Git fréquents	Élevée	Moyen	Un dossier par module, branches courtes, PR petites et fréquentes, `rebase` régulier
Retard d'un module bloquant les autres (M1, M3)	Moyenne	Élevé	Livrer M1 et le modèle `Task` en S1-S2 ; les autres travaillent avec des mocks en attendant
Changements du schéma de base de données mal coordonnés	Élevée	Élevé	Responsable unique du schéma, annonce préalable, migrations fusionnées vite
Temps réel incompatible avec l'hébergement choisi	Moyenne	Élevé	Décider dès S1 (service tiers type Pusher/Ably/Supabase), prototype en 1 jour
Intégration tardive révélant des incohérences	Élevée	Élevé	Intégration à chaque fin de sprint, démo commune
Faille de sécurité (accès à des données d'une autre équipe)	Moyenne	Élevé	Fonctions centralisées de permission, tests dédiés, revue de sécurité en S6
Surcharge d'un membre / abandon / maladie	Moyenne	Moyen	Documenter chaque module, binômage croisé, priorités P1/P2/P3 pour réduire le périmètre
Dépassement des quotas gratuits (BDD, stockage, temps réel)	Faible	Moyen	Surveiller la consommation, limites de taille, nettoyage des données de test
Interface incohérente d'un module à l'autre	Élevée	Moyen	Design system partagé, maquettes validées, relecture UI en revue de sprint
Fonctionnalités trop ambitieuses (P3 avant P1)	Moyenne	Moyen	Respect strict de l'ordre de priorité, validation du MVP avant tout bonus

14. Critères de validation

14.1 Recette fonctionnelle (scénario de démonstration)

1. Créer un compte, se connecter, modifier son profil. *(M1)*
2. Créer une équipe et inviter deux personnes par lien. *(M1)*
3. Créer un projet, personnaliser les colonnes. *(M2)*
4. Créer 5 tâches, les assigner, ajouter étiquettes, sous-tâches et échéances. *(M3)*
5. Déplacer des cartes par drag & drop, vérifier que l'autre navigateur se met à jour. *(M2)*
6. Retrouver ces échéances dans le calendrier, créer un jalon. *(M4)*
7. Échanger des messages à deux dans le chat, voir l'indicateur de saisie. *(M5)*
8. Déposer un fichier, le joindre à une tâche, le prévisualiser, le télécharger. *(M6)*
9. Consulter les statistiques, générer et exporter le rapport PDF. *(M7)*
10. Vérifier qu'un observateur ne peut rien modifier et qu'un non-membre n'accède à rien.

14.2 Barème indicatif de qualité

Critère	Poids indicatif
Fonctionnalités P1 complètes et fonctionnelles	30 %
Qualité du code, architecture, tests	20 %
Intégration réussie des 7 modules	15 %
Sécurité et gestion des permissions	10 %
Ergonomie, design, accessibilité	10 %
Documentation et rapport	10 %
Soutenance et démonstration	5 %

(À ajuster selon les consignes de votre enseignant.)

---

15. Annexes

15.1 Liste indicative des routes et Server Actions

Module	Actions / routes principales
M1	`register`, `login`, `logout`, `updateProfile`, `createTeam`, `inviteMember`, `acceptInvitation`, `updateMemberRole`, `removeMember`, `deleteTeam`, `markNotificationRead`
M2	`createProject`, `updateProject`, `archiveProject`, `createColumn`, `renameColumn`, `reorderColumns`, `deleteColumn`, `moveTask`
M3	`createTask`, `updateTask`, `deleteTask`, `assignUser`, `createLabel`, `toggleSubtask`, `addComment`, `listTasks`, `listMyTasks`
M4	`listCalendarEvents`, `createMilestone`, `updateDueDate`, `GET /api/calendar/[token].ics`
M5	`sendMessage`, `editMessage`, `deleteMessage`, `markChannelRead`, `listMessages(cursor)`, `POST /api/realtime/auth`
M6	`POST /api/upload`, `getDownloadUrl`, `renameFile`, `moveFile`, `deleteFile`, `createFolder`
M7	`getProjectStats`, `getMemberWorkload`, `getBurndown`, `generateReport`, `exportCsv`

15.2 Variables d'environnement (`.env.example`)

```
DATABASE_URL=
AUTH_SECRET=
AUTH_GOOGLE_ID=
AUTH_GOOGLE_SECRET=
AUTH_GITHUB_ID=
AUTH_GITHUB_SECRET=
NEXT_PUBLIC_APP_URL=
# Temps réel (selon le service choisi)
PUSHER_APP_ID=
PUSHER_KEY=
PUSHER_SECRET=
PUSHER_CLUSTER=
# Stockage de fichiers
STORAGE_BUCKET=
STORAGE_ACCESS_KEY=
STORAGE_SECRET_KEY=
# E-mails (réinitialisation, invitations)
EMAIL_SERVER=
EMAIL_FROM=
```

15.3 Glossaire

Terme	Définition
Kanban	Méthode visuelle de suivi du travail en colonnes représentant des étapes
WIP limit	Nombre maximum de tâches simultanées autorisées dans une colonne
Burndown chart	Graphique du travail restant au fil du temps
Vélocité	Quantité de travail terminée par unité de temps
Jalon (milestone)	Date clé du projet (rendu, soutenance)
Server Action	Fonction exécutée côté serveur et appelable depuis un composant React
Fractional indexing	Technique d'ordonnancement permettant d'insérer un élément entre deux autres sans renuméroter les autres
Mise à jour optimiste	Mise à jour de l'interface avant la confirmation du serveur, avec retour arrière en cas d'échec
Soft delete	Suppression logique : la donnée est marquée supprimée mais conservée en base
RGPD	Règlement général sur la protection des données
DoD	Definition of Done, critères de fin d'une fonctionnalité